Trust Juggler — 10-Hour Role Division

Oct 17, 2026 · @Deep

Ground Rules

The one contract to freeze in hour 1, and never touch again: the session event schema — the JSON shape the driver app sends and the backend ingests.


```
{
 "driver_id": "string",
 "session_id": "string",
 "timestamp": "ISO8601",
 "device_id": "string",
 "ip_subnet": "string",
 "zone": "string",
 "gps": [{"lat": 0.0, "lng": 0.0, "t": "ISO8601"}],
 "accel": [{"x": 0.0, "y": 0.0, "z": 0.0, "t": "ISO8601"}],
 "gyro": [{"x": 0.0, "y": 0.0, "z": 0.0, "t": "ISO8601"}],
 "touch_events": [{"type": "tap|swipe", "x": 0.0, "y": 0.0, "pressure": 0.0, "area": 0.0, "duration_ms": 0, "t": "ISO8601"}],
 "accept_latency_ms": 0
}
```


Why this is non-negotiable: every role can work against this schema without waiting on another role's code. ML writes scorers that consume it; backend writes an ingest endpoint that validates it; frontend mocks it to build the dashboard before any real data exists. Changing it mid-build forces all three to stop and re-sync.

No role blocks on another past hour 1:

- ML develops and tests scorers against synthetic sessions matching the schema — doesn't need the real API.
- Backend builds the ingest/scoring API against a mocked ML response matching the scorer's expected output shape — doesn't need real models yet.
- Frontend builds the dashboard against a mocked API response matching the API's expected output shape — doesn't need a real backend yet.

All three converge at the hour-5 integration gate (see Sync Points below), where mocks get swapped for the real thing one at a time.

Sync Points

[embed: node/b7e77e42-5c07]

Three gates matter more than the exact hour markers: the schema freeze (end of hour 1), the integration gate (end of hour 5, when mocks get swapped for real services), and the feature freeze (end of hour 8.5, after which only bug fixes and demo prep happen).

ML Workstream

Hour	Task	Deliverable	Tools
0–1	Agree the session schema with backend; download H-MOG and ODIDMobTraj; sketch the operational, motion and touch scorer interfaces (input shape → output: a 0–1 anomaly score)	Scorer function signatures, matching the frozen schema	H-MOG, ODIDMobTraj
1–3	Build the operational anomaly scorer: IsolationForest on synthetic rider features (acceptance rate, zone, timing)	score_operational(session) -> float returning a real number on synthetic input	PyOD
3–5	Build the motion scorer: LSTM autoencoder pretrained on H-MOG's accelerometer/gyroscope sessions, adapted with a vibration filter; build the synthetic rider simulator seeded from ODIDMobTraj priors	score_motion(session) -> float; a simulator that generates N synthetic sessions per rider persona	Keras/TF autoencoder reference architecture, H-MOG
5–7	Wire real scorer output into the API (replace the mocked response from hour 1–5); start the red-team simulator: lazy renter, slow-drift renter	Real scores flowing end-to-end for 2 of 4 attacker types	—
7–8.5	Finish the red-team simulator: split-shift sharer, phone-	All 4 attacker types + both honest-rider edge cases produce a session stream	—

	change+city-move (the honest-rider trap that must NOT trigger)		
8.5–9.5	Tune scorer thresholds against the red-team output; wire the GPT-4o/Claude call that generates the reason-vector explanation for any session crossing the challenge threshold	Calibrated thresholds; reason-vector text appears in escalated sessions	GPT-4o or Claude API
9.5–10	Bug fixes only; hand final scorer behavior to backend/frontend for the demo rehearsal	Frozen, demo-ready scorers	—

Don't attempt in 10 hours: real on-device SDK sensor collection, a trained touch-biometric model beyond a simple one-class classifier on H-MOG features, or any claim of a pretrained "rider identity" model — none exists publicly (see the main doc's ML Pipeline section).

Backend Workstream

Hour	Task	Deliverable	Tools
0–1	Agree the session schema with ML; set up repo, API keys, DB schema (sessions, riders, baselines table)	Repo scaffold + migrations applied	FastAPI / Express (per team's stack), Postgres
1–3	Build POST /v1/sessions/ingest validating against the frozen schema, storing raw sessions; build the context-change detector (device/IP/zone rule checks)	Ingest endpoint live against a local DB; context score computed per session	—
3–5	Build the two-axis combiner (context score × mocked behavior score → quadrant); stand up GET /v1/drivers/{id}/score and GET /v1/fleet/scores against mocked ML output	Combiner returns a quadrant + trust score on mocked input	—
5–7	Swap the mocked ML response for ML's real scorer output (integration gate); add the	End-to-end: real ML scores → combiner → API	—

	dual-timescale baseline (fast baseline + frozen identity anchor per rider)		
7–8.5	Add basic conformal threshold calibration per rider; build POST /v1/drivers/{id}/declare-change (rider-declared context change fast lane); build POST /v1/webhooks/alert stub	All 5 endpoints live; calibration applied to at least one rider cohort	—
8.5–10	Bug fixes; seed demo data for all 4 red-team scenarios + 2 honest-rider edge cases so the frontend demo has real data to show	Seeded, demo-ready database	—

Don't attempt in 10 hours: real authentication/authorization hardening, horizontal scaling, or a production-grade webhook retry system — state these as roadmap items in the pitch, not as built.

Frontend Workstream

Hour	Task	Deliverable	Tools
0–1	Agree the API response schema with backend; wireframe the admin dashboard: fleet quadrant view, per-driver trust timeline, review queue, cohort fairness panel	Wireframe/Figma or plain sketch of all 4 views	—
1–3	Build the dashboard shell and routing; build the fleet quadrant view against a hardcoded mock response matching the agreed schema	Quadrant chart renders with mock data	React/Next.js, a charting lib (Visualize/Recharts/D3)
3–5	Build the per-driver trust-score timeline view; build the review queue list view, both against mock data	Both views render with mock data	—
5–7	Swap mock data for the real backend API (integration gate); verify the quadrant updates live as sessions stream in	Dashboard shows real backend data	—

7–8.5	Build the "rent this account" toggle (calls the backend's simulated-handoff endpoint, or a frontend-only simulated session swap if backend doesn't expose one); build the cohort fairness panel	Live demo of a quadrant shift when toggled	—
8.5–10	Build the red-team scenario picker (run all 4 attacker types + 2 honest-rider cases side by side); polish UI; build the pitch deck/slides	Demo-ready dashboard + slides	—

Don't attempt in 10 hours: the actual driver mobile app or SDK — the admin dashboard is the deliverable; the driver-app side stays a diagram and a sentence in the pitch, not working code.

Risk Plan: What to Cut First

If any workstream is behind at the hour-5 integration gate, cut in this order — each cut preserves a working, honest demo rather than a broken ambitious one:

1. **Cut first:** the motion/touch LSTM autoencoder. Fall back to the operational IsolationForest scorer alone as the "behavior" axis. The two-axis story still works, just with one fewer signal — say so plainly in the pitch rather than faking motion data.
2. **Cut second:** conformal calibration. Use a fixed threshold instead of a per-rider calibrated one. Mention the calibration approach as designed-but-not-wired in the pitch.
3. **Cut third:** two of the four red-team attacker types. Keep the slow-drift renter and the phone-change+city-move honest-rider trap — that pair is the single comparison that proves the architecture (same context-change magnitude, opposite verdict).
4. **Cut fourth:** the webhook/alert endpoint and the "declare context change" fast lane. Both are easy to describe as roadmap items without live code.
5. **Never cut:** the two-axis combiner and the live dashboard quadrant view — that single picture is what makes the demo land. If backend integration is at risk, have frontend fall back to a scripted mock sequence that still visibly reacts, rather than showing nothing live.

Communication rule: whatever gets cut, say so explicitly in the demo and the pitch deck. A judge trusts a team that names its own gaps more than one that quietly papers over them.